

CodeDocGen—Exploring Automated Code Documentation Using Transformers

Matthew Presti
University of Colorado
Boulder, CO USA

Abstract

This project explores automated generation of code documentation for Python functions using transformer-based models. I implemented and evaluated CodeBERT for automatic docstring generation using the CodeSearchNet dataset, treating code documentation as a sequence-to-sequence translation task. Evaluation using ROUGE metrics and parameter identification accuracy shows moderate success (ROUGE-1: 0.24, parameter accuracy: 0.17). Despite computational constraints limiting training data and epochs, this framework demonstrates the potential of transformer models for code understanding. Several challenges were highlighted by this process such as obtaining coherent documentation from code structure alone. The results provide insights into how models interpret programming syntax as well as the resources required for developing improved automated documentation tools using transformers.

Introduction

Documentation is a critical component of software development that enhances code readability, maintainability, and knowledge

transfer. Despite its importance, documentation often remains neglected due to time constraints and preferences for writing code rather than explaining it. Automated code documentation aims to address this gap by generating natural language descriptions without manual effort.

This project focuses on generating docstrings for Python functions – concise explanations of a function's purpose, parameters, and return values. The task is particularly interesting because it requires translating between a formal programming language and natural language while preserving semantic meaning.

My primary goal was to explore training a transformer model from Hugging Face on the CodeSearchNet dataset and assess its usefulness towards implementing a system that automatically generates documentation. Additionally, I sought out to evaluate performance using both standard NLP metrics and practical usefulness measures.

Related Work

Several recent approaches have tackled code documentation using various NLP techniques. Kotsiantis et al. (2024) explored how transformer-based models enhance programming tasks, demonstrating that

transformer architectures effectively represent contextual relationships within code fragments. This informed my choice of CodeBERT as the primary model.

Hu et al. (2022) investigated the correlation between automated evaluation metrics and human assessment of documentation quality, revealing that metrics like BLEU and ROUGE often fail to capture aspects like accuracy and completeness that developers value. This guided my decision to include parameter identification as a practical usefulness metric.

Ahmad et al. (2020) presented a transformer-based approach for generating natural language summaries of source code, showing how transformers' attention mechanisms better capture long-range dependencies in code. Their preprocessing techniques for the CodeSearchNet dataset provided some guidance on data preparation.

Khan and Uddin (2022) explored using GPT-3 for code documentation across multiple programming languages. Although I ultimately used CodeBERT instead of a general language model due to computational constraints, their findings about the strengths of different model types informed my understanding of the task's challenges.

Data

This project utilizes the CodeSearchNet dataset, focusing exclusively on the Python subset containing 412,178 functions with associated docstrings. These functions represent real-world code from open-source

GitHub repositories, averaging 26.7 lines of code with docstrings averaging 289.0 characters in length. Initial data exploration revealed a moderate positive correlation (0.53) between code length and documentation length, though with significant variability.

The preprocessing workflow included filtering out functions without docstrings, cleaning the code and docstrings to remove special characters, tokenizing using CodeBERT's tokenizer, and splitting the data into training (80%), validation (10%), and test (10%) sets.

I performed the model training locally on my M2 Macbook Air and was not able to utilize GPU acceleration. Due to these computational constraints, I used only 100 samples for model training and evaluation. This limitation significantly affects the model's ability to learn complex patterns but allowed for reasonable experimentation within the project's scope and available resources.

Methodology

My approach treats code documentation generation as a sequence-to-sequence translation problem—a common encoder-decoder based machine learning approach that is well suited for translation and summarization. The final implementation differs from my initial proposal, focusing solely on CodeBERT rather than comparing multiple models due to computational and time constraints.

Model Architecture: I used CodeBERT through HuggingFace's transformers library. CodeBERT is a pre-trained transformer model specifically designed for programming language understanding that is based on the BERT architecture but pre-trained on both natural language and programming language data. I implemented an encoder-decoder architecture using CodeBERT as both components, allowing the model to encode Python function code into a contextualized representation and decode this representation into natural language documentation.

Training Process: The process involved loading and preprocessing the CodeSearchNet dataset, configuring the model with appropriate generation parameters, fine-tuning on the training subset using cross-entropy loss, and monitoring performance on a validation set. My python script utilized a custom CodeDocDataset class that efficiently packaged tokenized inputs and labels for the training loop. Training parameters included a learning rate of $5e-5$, batch size of 2 (limited by GPU memory), 3 training epochs, and AdamW optimizer with weight decay of 0.01. The script incorporated Hugging Face's Seq2SeqTrainer for streamlined training as it automatically handles gradient updates, validation steps, and model checkpointing.

Generation and Evaluation: The documentation generation phase involved extensive experimentation with decoding parameters. I iteratively refined the generation strategy in my evaluation script, starting with basic beam search and

gradually introducing more sophisticated techniques. For evaluation, I developed a two-pronged approach that assessed both linguistic similarity to human references through ROUGE metrics and functional correctness via parameter identification. This custom evaluation pipeline used regular expressions to extract function parameters from source code and measured how successfully the generated documentation mentioned these critical elements, providing insights into the functional utility of these models that NLP metrics alone don't capture.

Results

The evaluation results show limited success in generating documentation that captures some aspects of the original docstrings. Overall, the generated docstrings were not of usable quality and are mostly nonsensical. As shown in figure 1, ROUGE-1 (unigram overlap) achieved the highest score at 0.24, indicating that about 24% of individual words in the reference documentation appear in the generated documentation. The lower ROUGE-L score (0.10) suggests challenges in maintaining proper word sequences and coherent sentence structure. Only around 17% of function parameters were correctly identified in the generated documentation.

Here's an example showing the disparities between reference and generated documentation:

Function Declaration:

```
def findreplaceables(Class, parent,  
set=None,**kwargs):
```

Reference Documentation: "Internal method to find replaceable elements. Auxiliary function used by :meth:AbstractElement.replace. Can be overridden for more fine-grained control."

Generated Documentation: "to the. of,: an a" for (as` is and, not itm': will this string elementclass if all)] * that set The get specifiedSs None class in Arg byC method " children list child returned instanceFor elements internal argument nxml itemT This at } TheElement selectednotTo butttext"

The generated text contains relevant terms (e.g., "elements," "internal," "method") but is otherwise totally nonsensical. This captures the complex nature of proper decoder configuration and of coherent text generation in general. Given the sample size for training was 100, I am considering this a moderately successful result and suspect it would perform better with training performed on the entire dataset. See figure 2 for expanded docstring generation examples.

Discussion

The results demonstrate both potential and limitations of using transformer-based models for code documentation with limited resources.

Strengths: The model learned some association between code structures and documentation language, as evidenced by

specific terms appearing in the generated text. ROUGE-1 scores of 0.24 indicate some overlap with reference documentation, showing the model captured some relevant vocabulary. The implementation successfully integrated the full pipeline from data processing to evaluation.

Challenges and Limitations: Training data limitations severely restricted the model's ability to learn diverse code patterns and documentation styles. The generated documentation lacks grammatical structure and coherence, producing text with relevant keywords but in a disorganized manner and containing many randomly positioned characters. This speaks to the complexity of this task in the scope of natural language processing. Low parameter accuracy suggests the model struggles to connect function signatures with their natural language descriptions. Computational constraints restricted batch sizes and training epochs, affecting model convergence. Code contains specialized tokens and structures that differ from natural language, creating tokenization challenges as evidenced by the example result.

The incoherent nature of the generated documentation likely stems from insufficient training data and epochs, preventing the model from learning proper language structure while simultaneously trying to understand code semantics. Another factor is the variability of documentation styles in the dataset, as Python docstrings follow various conventions, making it difficult for a model with limited training to generalize effectively.

Conclusion and Future Work

This project explored automated Python function documentation generation using CodeBERT as a sequence-to-sequence translation problem. Despite moderate ROUGE-1 scores, the generated documentation exhibits significant coherence issues and very limited parameter identification accuracy, highlighting the challenges of this NLP task under computational constraints.

The experience provided valuable insights into applying transformer models to code understanding tasks and the complexities of translating between programming and natural languages. Furthermore it highlighted the resource intensive and complex nature of building useful systems or software with transformer architecture. Even with limited resources, a complete pipeline was implemented, though significant improvements would be needed for practical application.

Future work would focus on running this pipeline with increased computational resources, ideally performing training and evaluation on the entire CodeSearchNet dataset.

While this implementation achieved limited success in generating coherent documentation, it establishes a foundation for future exploration and provides insight to the utility of combining the CodeSearchNet

corpus with codeBERT model for this task. With additional resources and refinements, there is a legitimate opportunity to develop a sophisticated automated code docstring generator using transformer technology.

References

- [1] Kotsiantis, S., Verykios, V., & Tzagarakis, M. (2024). AI-assisted programming tasks using code embeddings and transformers. *Electronics*, 13(4), 767.
- [2] Hu, X., Chen, Q., Wang, H., Xia, X., Lo, D., & Zimmermann, T. (2022). Correlating automated and human evaluation of code documentation generation quality. *ACM Transactions on Software Engineering and Methodology*, 31(4), 1-39.
- [3] Ahmad, W. U., Chakraborty, S., Ray, B., & Chang, K. W. (2020). A transformer-based approach for source code summarization. *arXiv preprint arXiv:2005.00653*.
- [4] Khan, J. Y., & Uddin, G. (2022). Automatic code documentation generation using GPT-3. *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 1-12.
- [5] Barone, A. V. M., & Sennrich, R. (2017). A parallel corpus of Python functions and documentation strings for automated code documentation and code generation. *arXiv preprint arXiv:1707.02275*.

Final Results

Figure 1: Evaluating CodeBERT model on documentation generation

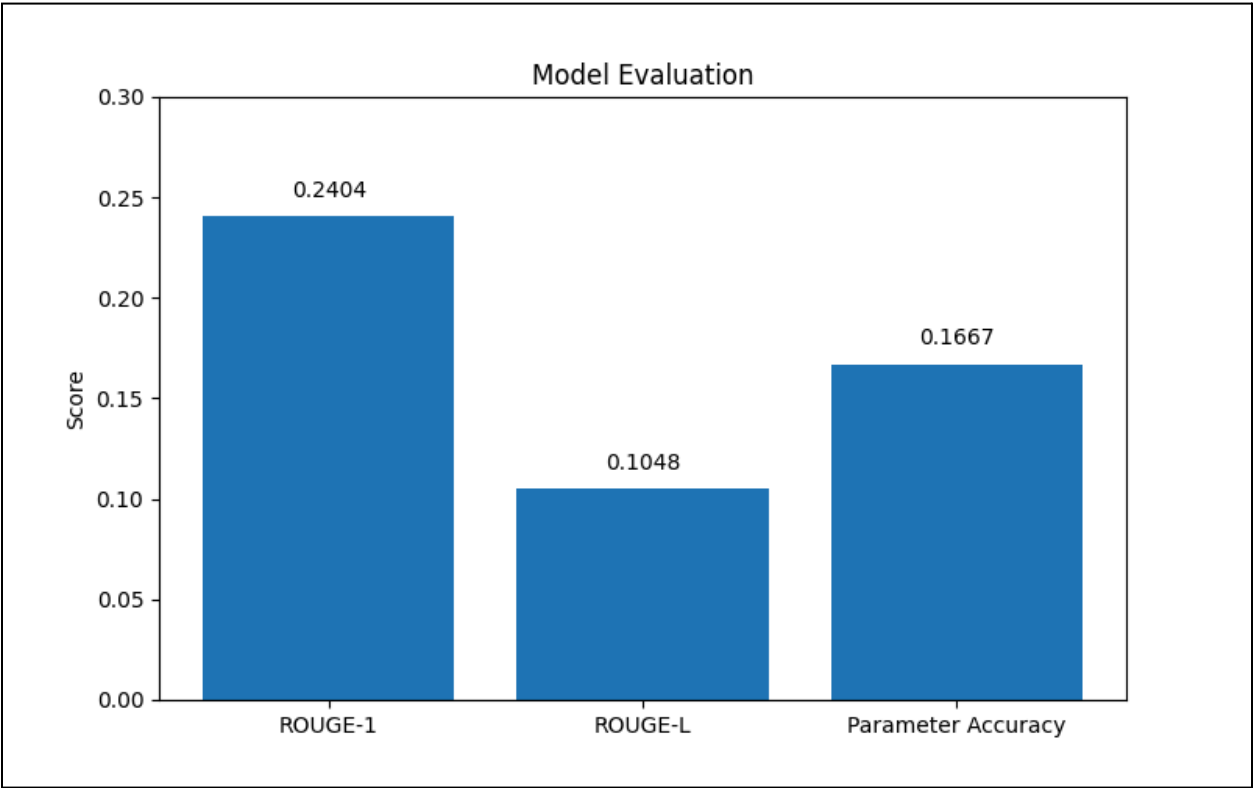


Figure 2: Docstring Generation Examples

Function #1: `def findreplaceables(Class, parent, set=None,**kwargs):`
 `"""Internal method to find replaceab...`

Reference: Internal method to find replaceable elements. Auxiliary function used by
:meth:`AbstractElement.replace`. Can be overridden for more fine-grained control.

Prediction: to the. of.; an a"for (as` is and, not itm': will this string elementclass if all)] * that set The get
specifiedSs None class in Arg byC method " children list child returned instanceFor elements internal
argument nxml itemT This at } TheElement selectednotTo butttext

Function #2: def morphemes(self,set=None):

Reference: Generator yielding all morphemes (in a particular set if specified). For retrieving one specific morpheme by index, use morpheme() instead

Prediction: ∴ the` of,m (a this for to that will get an,class and" by not it is) if class specified method as TheT' set at None in all stringFor * returnedS element.] " lists noReturnsC]You Arg ThisTo returns argument itemxml child based i do youElement selected

Function #3: def words(self, index = None):

Reference: Returns a generator of Word elements found (recursively) under this element.

Arguments:

* ``index``: If set to an integer, will retrieve and return the n'th element (starting at 0) instead of returning the list of all.

Prediction: ∴ to, the a class (is for an of, as:` this and notmclass) get will NoneS all specified that" The child list in by' element itElement string internal if set methods or]For] returned s at argument *T instance basedxml " n ThisYou`` selected Arg item2The children.C
